Innovative Evaluation Practices for BCSE308P -Computer Networks Lab

Fall Semester 2026-27

INTELLIGENT ROUTING PROTOCOL: ADAPTIVE MULTI-METRIC ROUTING FOR DYNAMIC NETWORK CONDITIONS

24BCE2430 THIPIREDDY BHUVANA KRUTHI

24BCE2431 NEHA DAMANI

24BCE2514 AFREEN FAIZAL

B.Tech.

in

Computer Science and Engineering

School of Computer Science and Engineering



October 2026

ABSTRACT

Conventional interior routing protocols such as OSPF and RIP choose paths using a static metric, either an administratively configured cost or a hop count, and change routes only when a link is declared down. As a result they keep forwarding traffic over links that are still up but congested, lossy or unstable. This project designs, implements and evaluates an Intelligent Routing Protocol (IRP) that selects paths from live, measured link conditions. Every 0.5 s a link monitor probes each link and maintains EWMA-smoothed estimates of latency, packet loss, load and capacity, together with a decaying flap score. A composite link cost, expressed in effective milliseconds, combines the expected delay (including the queueing delay predicted with the flow's own demand added), a loss penalty of $-\ln(1-p)$ that makes loss additive along a path, a bandwidth-headroom penalty and a route-flap penalty. Dijkstra's algorithm computes the least-cost path, while 15 % hysteresis, a 3 s hold-down, flap damping, own-load subtraction and early failure detection keep routes stable.

The protocol was implemented in Python as a discrete-time, flow-level network simulator and compared with OSPF-like static routing, latency-only dynamic routing and RIP-like hop-count routing over nine scenarios, each repeated with ten random seeds and tested with Welch's t-test. Against OSPF-like routing, IRP delivered a significantly higher fraction of packets in all seven loss, capacity, failure, congestion and flapping scenarios (for example 0.996 vs 0.867 under a bandwidth drop and 0.996 vs 0.938 on a flapping link), recovered from a link failure about nine times faster (0.13 s vs 1.15 s) and behaved identically on a healthy network. The price is 1 to 8 ms of extra mean latency in four scenarios, where IRP deliberately prefers a slightly longer path that does not lose packets.

Certificate

This is to certify that the project report titled "**Intelligent Routing Protocol: Adaptive Multi-Metric Routing for Dynamic Network Conditions**" is a bonafide record of the work done by **Thipireddy Bhuvana Kruthi (24BCE2430), Neha Damani (24BCE2431) and Afreen Faizal (24BCE2514)** in partial fulfilment of the requirements of the Innovative Evaluation Practices for BCSE308P - Computer Networks Lab during the Fall Semester 2026-2027.

Name and Signature of the Students

THIPIREDDY BHUVANA KRUTHI (24BCE2430) _____

NEHA DAMANI (24BCE2431) _____

AFREEN FAIZAL (24BCE2514) _____

TABLE OF CONTENTS

Sl.No	Contents	Page No.
	Abstract	2
1.	**INTRODUCTION**	6
	1.1 Background and motivation	6
	1.2 Problem statement	6
	1.3 Objectives	7
	1.4 Scope and limitations	7
	1.5 Mapping to course modules and the lab experiment	7
2.	**LITERATURE SURVEY**	9
	2.1 Overview of the existing protocol or scheme	9
	2.2 Review of related work	9
	2.3 Research gap and justification of novelty	10
3.	**PROPOSED SYSTEM**	11
	3.1 System architecture and topology	11
	3.2 Baseline mechanism description	12
	3.3 Proposed mechanism: algorithm, pseudocode, flowchart	12
	3.4 Novelty statement	14
	3.5 Assumptions and design decisions	15
4.	**IMPLEMENTATION**	16
	4.1 Tools, platform, versions and hardware	16
	4.2 Simulation setup (topology, configuration, parameter table)	16
	4.3 Code structure and explanation of key modules	17
	4.4 Challenges faced and how they were solved	18
5.	**EXPERIMENTAL SETUP AND PERFORMANCE EVALUATION**	19
	5.1 Performance metrics and why they were chosen	19
	5.2 Scenarios and parameter values	19
	5.3 Results: baseline vs proposed (mean of repeated runs)	19
	5.4 Analysis and discussion	22

5.5 Critical evaluation 23

6. CONCLUSION AND FUTURE WORK 24

6.1 Summary of work and key findings 24

6.2 Limitations 24

6.3 Future enhancements 24

7. REFERENCES 26

References (IEEE style) 26

Appendix A: Important source code and configuration listings 28

Appendix B: Additional screenshots and raw result tables 30

1. INTRODUCTION

1.1 Background and Motivation

Routing is the network-layer function that decides which sequence of routers a packet traverses from its source to its destination. Interior gateway protocols perform this task inside an autonomous system. The two classic families are link-state protocols such as OSPF [1], in which every router learns the full topology and runs Dijkstra's shortest-path algorithm [3], and distance-vector protocols such as RIP [2], in which routers exchange distance estimates with their neighbours using the Bellman-Ford principle.

Both families describe each link with a fixed number. OSPF uses an administratively configured cost that is normally derived from the interface bandwidth, and RIP simply counts hops. These numbers do not change when the link becomes congested, starts dropping packets or keeps going up and down. The protocols react only when a link is declared completely dead, which takes several missed hello messages. Real links, however, often degrade without failing: a failing optic or wireless interference raises the loss rate, cross traffic fills a queue, or a link renegotiates to a lower speed. During such events a shortest-path protocol keeps sending traffic over a link that is technically alive but is delivering poor service.

The motivating example used throughout this project is a four-router network in which the path A→B→D has 20 ms of latency but loses 15 % of packets, while A→C→D has 35 ms of latency and loses about 1 %. A shortest-path protocol chooses A→B→D because 20 ms is less than 35 ms, and therefore loses 15 of every 100 packets. Paying 15 ms of extra delay to avoid that loss is clearly the better choice, but a protocol can only make it if it measures the loss and weighs it against delay. This gap between "shortest" and "best" is the motivation for an intelligent, condition-aware routing protocol.

1.2 Problem Statement

Design, implement and evaluate a routing protocol that selects and updates paths from measured link conditions (latency, packet loss, available bandwidth and stability) instead of a static distance metric. The protocol must move traffic away from degraded links faster than failure-driven protocols, must not oscillate between paths when conditions are noisy or when its own traffic changes the load, and must not perform worse than the baseline on a healthy network. The improvement must be demonstrated quantitatively against an OSPF-like baseline under repeatable fault scenarios, with repeated runs and statistical significance tests.

1.3 Objectives

1. Build a measurement layer that probes every link periodically and estimates latency, loss, load and capacity from noisy samples, including dead-timer failure detection and a record of recent flaps.
2. Define a single composite link cost that combines delay, loss, available bandwidth and stability in one unit, so that Dijkstra's algorithm can be used unchanged.
3. Add stability mechanisms (hysteresis, hold-down, flap damping and own-load subtraction) so the protocol adapts quickly without route oscillation, and record a human-readable reason for every route change.
4. Implement OSPF-like, latency-only and hop-count baselines that share exactly the same monitor, so that comparisons isolate the effect of the routing logic.
5. Evaluate all policies over nine fault scenarios with ten seeds each, report mean ± standard deviation, and test significance with Welch's t-test.
6. Provide an interactive dashboard and an automated test suite so the results can be reproduced and demonstrated live.

1.4 Scope and Limitations

Scope. The project covers intra-domain (interior) routing of unicast flows in a single administrative domain. It includes link monitoring, cost computation, path selection, route-update control, a network simulator with fault injection, baseline protocols, a statistical evaluation framework, a Streamlit dashboard and 29 automated tests.

Limitations. The simulator is flow-level and discrete-time (100 ms ticks) rather than packet-level, so TCP congestion-control dynamics are not modelled and queueing follows an M/M/1-style approximation. Routes are computed per flow from a shared link-state view; the flooding of link-state advertisements and its convergence delay are not modelled beyond probe and detection delay. The cost weights were tuned by hand. The protocol has not been deployed on physical routers or in a packet-level emulator.

1.5 Mapping to Course Modules and the Lab Experiment

The project extends several topics of BCSE308P Computer Networks and the corresponding lab experiments, as summarised in Table 1.1.

Table 1.1: Mapping of the project to course topics and lab work

Course topic / lab experiment	How it is used in this project
Network layer: routing algorithms (link-state routing, Dijkstra's shortest path)	The routing engine runs Dijkstra over per-link costs, exactly as a link-state router computes its SPF tree. The OSPF-like baseline is the classic shortest-path behaviour.
Distance-vector routing (RIP, hop count)	The hop-count baseline reproduces RIP's metric, where every link costs 1.
Routing protocol mechanisms: hello / dead timers, failure detection	Every policy uses probes every 0.5 s and a dead timer of three missed probes, mirroring OSPF hello and dead intervals.
Congestion, queueing and quality of service (delay, throughput, loss)	Link cost includes M/M/1-style queueing delay and overflow loss; evaluation measures delivery ratio, throughput, mean and 95th-percentile latency.
Network performance measurement and analysis	Ten-seed experiments, mean ± SD, 95 % confidence intervals and Welch's t-test for every scenario.
Network simulation lab experiments	A custom discrete-time simulator with an 8-router, 13-link mesh topology and nine fault scenarios.

2. LITERATURE SURVEY

2.1 Overview of the Existing Protocol or Scheme

OSPF (Open Shortest Path First) [1] is the most widely used link-state interior gateway protocol. Each router floods link-state advertisements (LSAs) describing its links, builds an identical link-state database and runs Dijkstra's algorithm to compute a shortest-path tree. The cost of an interface is a static value, by default a reference bandwidth divided by the interface bandwidth. Neighbours exchange hello packets (10 s by default) and a neighbour is declared down when no hello arrives within the dead interval (40 s by default). OSPF therefore reacts only to topology changes: a link that is up but lossy or congested keeps its cost, and traffic keeps using it.

RIP (Routing Information Protocol) [2] is a distance-vector protocol that uses hop count as its only metric, with a maximum of 15 hops and periodic updates every 30 s. It ignores delay, bandwidth and loss entirely and converges slowly after failures. **EIGRP** [13] uses a composite metric of bandwidth and delay; its load and reliability terms exist but are disabled by default because they made routes unstable. This history shows that the difficulty with dynamic metrics is not computing them but keeping routing stable when they are used.

2.2 Review of Related Work

Table 2.1 summarises the work that most directly shaped the design of the proposed protocol.

Table 2.1: Review of related work

S.No	Authors, year	Technique	Metrics and tool	Merits	Gaps
1	Moy, 1998 [1]	OSPFv2 link-state routing, Dijkstra SPF	Static interface cost; RFC specification	Fast, loop-free, scalable, standard	Cost ignores loss and load; reacts only to up/down
2	Hedrick, 1988 [2]	RIP distance-vector routing	Hop count; RFC specification	Very simple to configure	No delay, bandwidth or loss awareness; slow convergence
3	Khanna & Zinky, 1989 [4]	Revised ARPANET delay-based metric with bounded, smoothed cost	Measured delay and utilisation; ARPANET deployment	Load-sensitive routing that damped oscillation	Delay only; no explicit loss or flap handling
4	Wang & Crowcroft, 1992 [5]	Analysis of shortest-path routing with dynamic metrics	Delay-based costs; analytical model and simulation	Explains why load-sensitive routing oscillates	Analysis only; no concrete damping design
5	Fortz & Thorup, 2000 [6]	Offline optimisation of OSPF link weights	Link utilisation; traffic-matrix optimisation	Large gains without changing OSPF	Offline; weights do not react to live faults

S.No	Authors, year	Technique	Metrics and tool	Merits	Gaps
6	De Couto et al., 2003 [7]	ETX path metric for multi-hop wireless	Probe-based delivery ratio; testbed	Loss-aware metric, additive along a path	Wireless-specific; ignores delay and capacity
7	Kandula et al., 2005 [8]	TeXCP online traffic engineering	Utilisation; ns-2 simulation	Responsive yet provably stable load balancing	Needs pre-computed tunnels; targets load only
8	Andersen et al., 2001 [9]	Resilient Overlay Networks (RON)	Probed latency, loss, throughput; Internet testbed	Recovers from outages within seconds	Overlay layer, not an IGP; per-application
9	Boyan & Littman, 1994 [10]	Q-routing (reinforcement learning)	Delivery time; simulation	Learns routes under changing load	Slow to converge; can be unstable
10	Villamizar et al., 1998 [11]	Route flap damping	Penalty with exponential decay; BGP	Suppresses unstable routes	Designed for BGP, not combined with link metrics

2.3 Research Gap and Justification of Novelty

The survey shows three recurring gaps. First, deployed interior protocols (OSPF, RIP) use static metrics and react only to complete failures, so they cannot avoid links that are alive but lossy, congested or flapping. Second, the dynamic schemes that do measure conditions usually focus on a single metric (delay in the ARPANET metric, loss in ETX, utilisation in TeXCP), so they miss faults that do not appear in that metric. In this project's own results the latency-only baseline fails exactly on loss and flapping faults. Third, the classic reason for avoiding dynamic metrics is oscillation [4], [5], and this is usually handled by one mechanism at a time rather than by a combination designed together with the cost.

The proposed protocol addresses these gaps by combining four live metrics in a single additive cost that Dijkstra can minimise exactly, by predicting the effect of the flow's own traffic instead of reacting to it, and by pairing the cost with hysteresis, hold-down, flap damping and early failure detection. It is evaluated against both a static (OSPF-like) and a single-metric dynamic (latency-only) baseline under identical measurement conditions, which shows where each part of the design matters.

3. PROPOSED SYSTEM

3.1 System Architecture and Topology

The system has five stages, shown in Fig. 3.1. The network simulator holds the true state of every router and link. The link monitor measures that state through periodic, noisy probes. The cost function turns each link's estimates into one number. Dijkstra's algorithm finds the cheapest path for each flow, and the stability stage decides whether the route should actually change. An important design point is that the routing engine never reads the true network state: it sees only what the monitor has measured, so its information is noisy and slightly late, as on a real network.

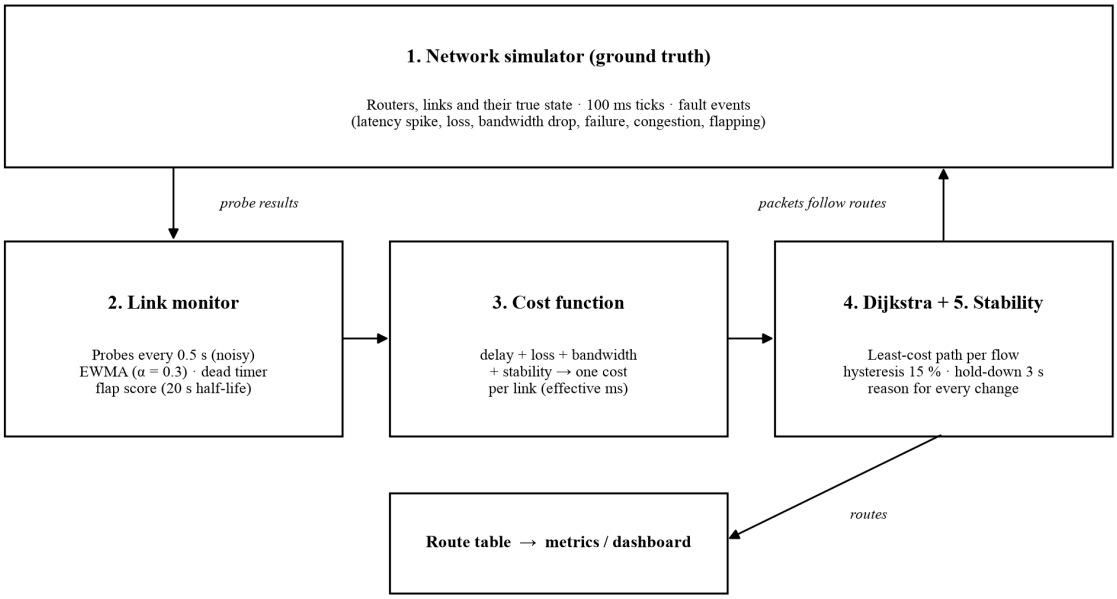
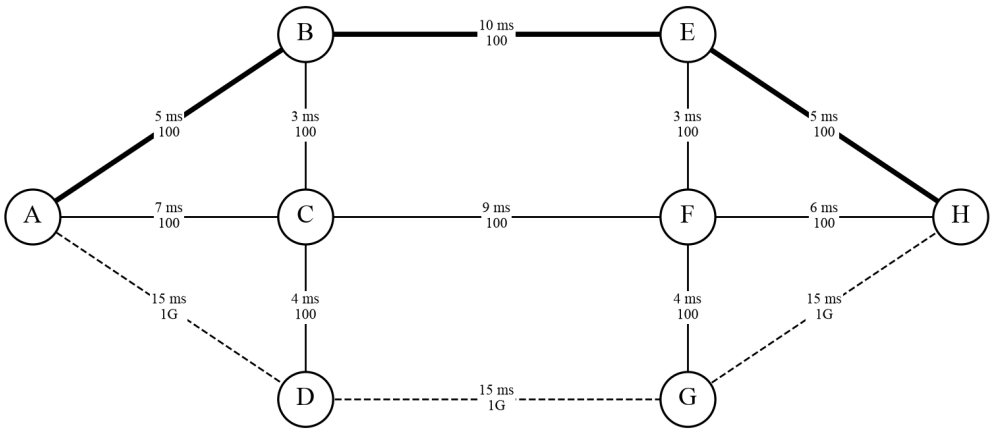


Fig. 3.1: Architecture of the Intelligent Routing Protocol

Most experiments use the 8-router, 13-link mesh shown in Fig. 3.2. Flow f1 from A to H at 40 Mb/s is the flow under test, and flow f2 from C to E at 30 Mb/s is competing traffic. Three disjoint-ish paths exist for f1: A→B→E→H (20 ms, the shortest), A→C→F→H (22 ms) and A→D→G→H (45 ms but 1 Gb/s). The motivating example uses a separate 4-router diamond (A→B→D with 15 % loss on B-D, A→C→D clean).



Thick line: shortest path of f1 (A→B→E→H, 20 ms). Dashed: 1 Gb/s long path (A→D→G→H, 45 ms).
Flows: f1 A→H 40 Mb/s (under test), f2 C→E 30 Mb/s (competing). Labels: latency, capacity (Mb/s).

Fig. 3.2: Test topology mesh8 with link latency and capacity

3.2 Baseline Mechanism Description

The primary baseline is **OSPF-like static routing**. Its link cost is the configured (nominal) latency of the link, which never changes with conditions. It reacts only when the monitor declares a link down after three consecutive missed probes (1.5 s), and when the link comes back. This mirrors OSPF's behaviour of using a static interface cost and recomputing the shortest-path tree only on topology change. Two further baselines are included to separate the effects of the design:

- **Latency-only dynamic routing**: cost = measured latency + predicted queueing delay, with 10 % hysteresis and a 2 s hold-down. It represents single-metric adaptive schemes.
- **Hop-count routing (RIP-like)**: every live link costs 1.

All four policies share the same link monitor (same probe rate, same noise, same dead timer), so any difference in results comes from the routing logic alone.

3.3 Proposed Mechanism: Algorithm, Pseudocode, Flowchart

(a) Link measurement

Every 0.5 s the monitor sends 50 probes over each link. Latency samples include 5 % Gaussian jitter and loss is sampled binomially. Estimates are smoothed with an exponentially weighted moving average, $\hat{x} \leftarrow 0.7 \cdot \hat{x} + 0.3 \cdot x$. A link is declared down after three consecutive missed probes; each down transition adds 1 to a flap score that halves every 20 s. Crucially, a missed

probe is also fed into the loss estimate as a sample of 100 % loss, so the loss term of the cost rises after a single missed probe.

(b) Composite link cost

For a flow with demand D on link l, the cost in effective milliseconds is given by Eq. (3.1):

$$C(l) = w_lat \cdot (d + q(\rho')) + w_loss \cdot (-\ln(1 - p')) + w_bw \cdot (B_ref / B_avail) + w_stab \cdot \varphi \qquad (3.1)$$

where d is the smoothed latency; $\rho' = (L - L_own + D) / B$ is the predicted utilisation with the flow's own smoothed load L_own removed and its demand D added; $q(\rho) = 3 \cdot \rho / (1 - \rho)$ ms is the M/M/1-style queueing delay (capped at 250 ms); $p' = 1 - (1 - p)(1 - o(\rho'))$ combines measured random loss p with overflow loss $o(\rho) = 1 - 1/\rho$ for $\rho > 1$; B_avail is the free capacity; and φ is the flap score. The default weights are w_lat = 1, w_loss = 300, w_bw = 1, w_stab = 25 and B_ref = 100 Mb/s.

The loss term uses $-\ln(1 - p)$ because the delivery probability of a path is the product $\Pi(1 - p_i)$ of its links, and the logarithm turns this product into a sum. Dijkstra adds link costs, so it then minimises the end-to-end loss exactly. On the motivating example this gives B-D a cost of 60.5 (48.8 of it from loss) against 20.8 each for A-C and C-D, so the protocol chooses A→C→D (41.5) over A→B→D (72.6).

(c) Route selection and update algorithm

```
Algorithm 1: IRP route update (after every probe round, for every flow f)
Input: monitor estimates, current path P, hysteresis h = 0.15,
       hold-down H = 3 s, current time t
 1  for each link l: own[f][l] <- EWMA(own[f][l], D_f if l in P else 0)
 2  for each link l: cost[l] <- INF if l is down, else C(l) from Eq. (3.1)
 3  (P*, C*) <- Dijkstra(src_f, dst_f, cost)
 4  if f has no route yet: install P* ("initial route"); return
 5  C <- sum of cost[l] over the links of P
 6  if P* = P: refresh baseline costs; return
 7  if C = INF: install P* ("link declared down"); return  // no hold-down
 8  if C* >= C: return
 9  if (C - C*) / C <= h: return                          // hysteresis
10  if t - last_change[f] < H: return                     // hold-down
11  reason <- link and metric whose cost grew most since P was installed
12  install P*; last_change[f] <- t; log (t, P, P*, reason)
```

The decision logic is also shown as a flowchart in Fig. 3.3. Dijkstra is implemented with a binary heap, so each route computation costs $O((V + E) \log V)$ per flow per probe round.

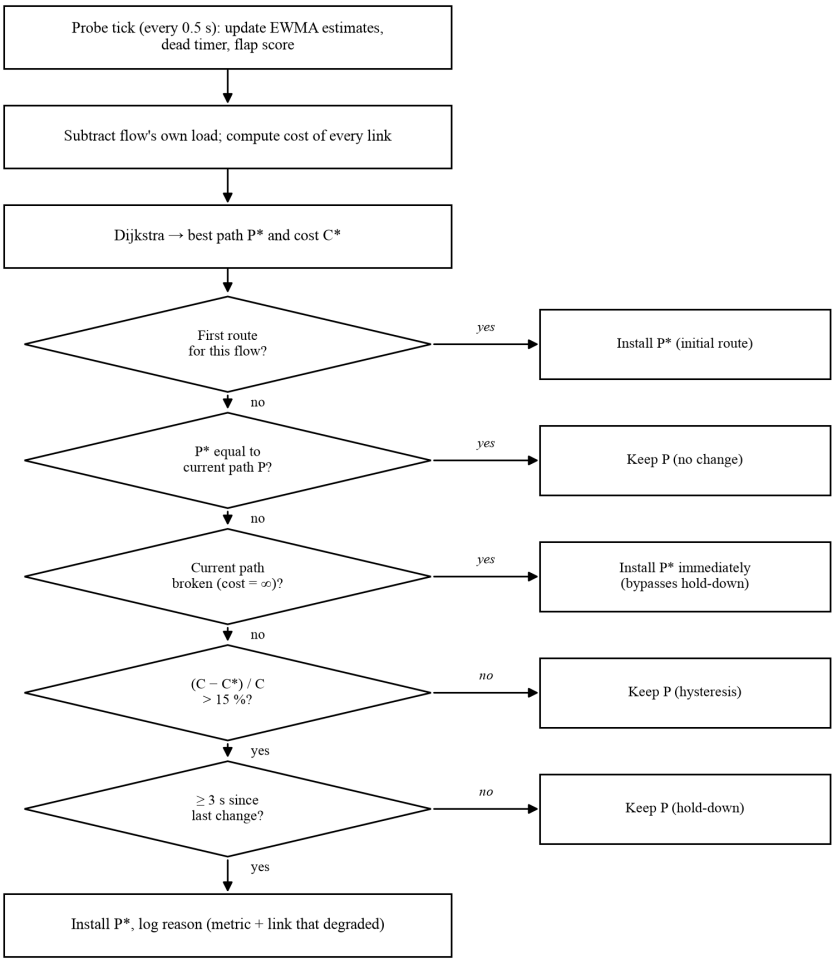


Fig. 3.3: Flowchart of the route-update decision

3.4 Novelty Statement

Table 3.1 lists exactly what differs between the OSPF-like baseline and the proposed protocol.

Table 3.1: Differences between the baseline and the proposed protocol

Aspect	Baseline (OSPF-like)	Proposed (IRP)
Link metric	Static configured latency	Live composite of delay, loss, free bandwidth and stability
Loss handling	Ignored	$-\ln(1-p)$ term, additive so Dijkstra minimises path loss exactly
Load awareness	None	Predicts queueing and overflow with the flow's demand added, before moving
Self-congestion	Not applicable	Flow's own load is subtracted, so it does not flee its own traffic
Failure reaction	After 3 missed probes (dead timer)	After 1 missed probe, via the loss estimate
Flapping links	Follows every up/down transition	Flap penalty with 20 s half-life keeps traffic away

Aspect	Baseline (OSPF-like)	Proposed (IRP)
Route stability	Changes only on topology change	15 % hysteresis and 3 s hold-down for cost changes
Explainability	No reason recorded	Every change logs the link and metric that triggered it

3.5 Assumptions and Design Decisions

- **Shared measurement view.** Every router is assumed to have the same, slightly delayed, link-state view, as in a converged link-state protocol. Routes are computed per flow at the source.
- **One unit for all metrics.** Costs are expressed in effective milliseconds so the weights have a clear meaning: w_loss = 300 means that 1 % loss is worth about 3 ms of delay and 15 % loss about 49 ms.
- **M/M/1 queueing approximation** for predicting delay and overflow loss from utilisation, shared by the simulator and the cost function.
- **Failures bypass the hold-down**, because staying on a broken path is always worse than an extra route change.
- **Fair comparison.** All policies use the same probe interval (0.5 s), noise model and dead timer (3 probes); only the cost function and the stability parameters differ.
- **Determinism.** Every run is seeded, so any result in this report can be reproduced exactly.

4. IMPLEMENTATION

4.1 Tools, Platform, Versions and Hardware

Table 4.1: Software and hardware used

Item	Details
Programming language	Python 3.14.5 (works on Python 3.10 or newer)
Core protocol and simulator	Python standard library only (heapq, random, math, dataclasses)
Evaluation and statistics	pandas 3.0.6, SciPy 1.18.1 (Welch's t-test)
Plots and dashboard	Matplotlib 3.11.2, Plotly 7.1.0, Streamlit 1.65.0
Testing	pytest 9.1.1 (29 automated tests)
Operating system	macOS 27 (also runs on Linux and Windows)
Hardware	Apple M4 processor, 16 GB RAM
Editor and version control	Visual Studio Code, Git / GitHub

4.2 Simulation Setup (Topology, Configuration, Parameter Table)

The network is simulated in discrete time with 100 ms ticks. In each tick the simulator applies any scheduled fault events, runs a probe round when due, lets the routing engine update routes, and then transmits 100 packets per flow along its current path. A packet is delivered with probability $\Pi(1 - p_i)(1 - o_i)$ over the links of the path, and its latency is the sum of propagation and queueing delay with 2 % jitter. All links have a base random loss of 0.1 %. The topology is shown in Fig. 3.2 and the parameters in Table 4.2.

Table 4.2: Simulation and protocol parameters

Parameter	Value
Simulated time per run	60 s (fault active from 15 s to 40 s)
Simulator tick	100 ms; 100 packets per flow per tick
Probe interval / probes per round	0.5 s / 50 probes, 5 % latency jitter
EWMA smoothing factor α	0.3
Dead timer	3 consecutive missed probes (all policies)
Flap score half-life	20 s
Cost weights w_lat, w_loss, w_bw, w_stab	1, 300, 1, 25 (B_ref = 100 Mb/s)
Hysteresis / hold-down (IRP)	15 % / 3 s
Hysteresis / hold-down (latency-only)	10 % / 2 s
Queueing model	$q(\rho) = 3\rho / (1 - \rho)$ ms, capped at 250 ms
Topology	mesh8: 8 routers, 13 links (100 Mb/s; A-D-G-H path 1 Gb/s)
Traffic	f1: A→H 40 Mb/s (under test); f2: C→E 30 Mb/s
Repetitions	10 seeds per scenario per policy (360 runs in total)

4.3 Code Structure and Explanation of Key Modules

The source code is organised as a Python package (irp/) with a dashboard and a test suite. It is available at https://github.com/nehadamani0909/F2_Team18. Table 4.3 describes each file.

Table 4.3: Code structure

File	Contents
irp/topology.py	Network, Link and LinkState classes; queueing-delay and overflow-loss model
irp/monitor.py	LinkMonitor: probing, EWMA smoothing, dead timer, flap score
irp/policies.py	Adaptive cost function (IRP) and the OSPF-like, latency-only and hop-count baselines
irp/engine.py	RoutingEngine: Dijkstra, hysteresis, hold-down, own-load tracking, route-change reasons
irp/simulator.py	Discrete-time simulator and fault events (fail, restore, degrade)
irp/scenarios.py	Topologies (mesh8, diamond) and the nine experiments
irp/metrics.py	Delivery ratio, throughput, latency, recovery time and time degraded
irp/evaluation.py	Multi-seed runs, confidence intervals, Welch's t-tests, figures and REPORT.md
irp/__main__.py	Command-line interface: demo, scenarios, run, evaluate, dashboard
dashboard/app.py	Interactive Streamlit dashboard with live fault injection
tests/test_irp.py	29 pytest tests

LinkMonitor.probe() updates each link's estimate once per probe round. If the link is down it counts a missed probe, feeds a 100 % loss sample into the loss EWMA and, after three misses, declares the link down and increments the flap score. If the link is up it samples latency and loss and smooths them; a link that comes back starts from fresh measurements rather than stale ones.

AdaptivePolicy.link_cost() implements Eq. (3.1). It computes the predicted utilisation with the flow's own load removed, the queueing delay, the combined random and overflow loss, the free bandwidth and the flap penalty, and returns both the total and the per-metric breakdown used to explain route changes.

RoutingEngine.shortest_path() is a heap-based Dijkstra that skips links with infinite cost (down links). **RoutingEngine._update_flow()** implements Algorithm 1: initial install, failure bypass, hysteresis and hold-down. **_degradation_reason()** compares the current per-metric costs with the best values seen since the route was installed and names the link and metric that grew the most, for example "loss on B-E degraded: loss ≈30.2%, util 40% (73% cheaper)". The key listings are reproduced in Appendix A.

4.4 Challenges Faced and How They Were Solved

Table 4.4: Challenges and solutions

Challenge	Solution
Loss is multiplicative along a path, but Dijkstra adds costs	Used $-\ln(1 - p)$ as the loss term, which makes loss exactly additive
A flow fled from links made busy by its own traffic, then came back, oscillating	Track each flow's own load with the same EWMA as the monitor and subtract it before computing utilisation
Noisy probes caused frequent small route changes	EWMA smoothing, 15 % hysteresis and a 3 s hold-down for cost-driven changes
Dead-timer failure detection was as slow as OSPF	Count every missed probe as a loss sample, so the cost rises after one missed probe
Flapping links pulled traffic back each time they recovered	Flap score with a 20 s half-life added to the cost
Unfair comparisons if baselines measured differently	All policies share one LinkMonitor with identical probe rate, noise and dead timer
Results that could be due to chance	Ten seeds per scenario, mean ± SD, 95 % confidence intervals and Welch's t-test
Hard to explain why a route changed	Per-metric cost breakdown stored at install time; the largest increase is reported as the reason

5. EXPERIMENTAL SETUP AND PERFORMANCE EVALUATION

5.1 Performance Metrics and Why They Were Chosen

Table 5.1: Performance metrics

Metric	Definition	Why it was chosen
Delivery ratio	Packets delivered ÷ packets sent (whole run)	Direct measure of reliability
Throughput ratio	Data delivered ÷ data offered	Shows how much of the demand is served
Mean and p95 latency	Average delay, and the delay that 95 % of packets beat	Captures the cost of choosing longer paths and queueing
Recovery time	Seconds from fault onset until f1 is back to ≥ 90 % of normal throughput	Measures reaction speed
Time degraded	Seconds during the fault with f1 below 90 % of normal throughput	Total impact of a fault on users
Route changes	Number of route switches in the run	Measures stability and oscillation

5.2 Scenarios and Parameter Values

Nine scenarios were run, each for 60 s with the fault active between 15 s and 40 s unless stated otherwise (Table 5.2).

Table 5.2: Experimental scenarios

Scenario	Fault and parameter values
S1 Steady state	No faults; checks that adaptation costs nothing on a healthy network
S2 Latency spike	B-E latency 10 → 150 ms
S3 Bandwidth drop	E-H capacity 100 → 15 Mb/s (f1 needs 40 Mb/s)
S4 Packet loss increase	B-E random loss 0.1 % → 12 %
S5 Link failure	B-E down at 15 s, restored at 40 s
S6 Congestion build-up	B-E cross traffic 50 → 70 → 85 → 95 Mb/s at 15, 18, 21, 24 s
S7 Flapping link	E-H fails for 2 s every 4 s between 15 s and 40 s
S8 Random faults	7 random faults (failure, latency ×4–12, loss 5–25 %, capacity 5–30 %), 5–15 s each
S9 Lossy shortcut	Diamond: A→B→D 20 ms with 15 % loss vs A→C→D 35 ms with ~1 % loss; B-D repaired at 30 s

5.3 Results: Baseline vs Proposed (Mean of Repeated Runs)

Table 5.3 compares the proposed protocol with the OSPF-like baseline. Each value is the mean ± standard deviation over 10 seeds. Improvement is positive when the proposed protocol is better (higher delivery, or lower latency, recovery time, time degraded or route changes).

Table 5.3: Baseline (OSPF-like) vs proposed (IRP), mean ± SD over 10 seeds

Scenario	Metric	Baseline (mean ± SD)	Proposed (mean ± SD)	Improvement (%)
S1 Steady state	Delivery ratio	0.997 ± 0.000	0.997 ± 0.000	+0.0
	Mean latency (ms)	20.28 ± 0.02	20.28 ± 0.02	+0.0
S2 Latency spike	Delivery ratio	0.997 ± 0.000	0.997 ± 0.000	+0.0
	Mean latency (ms)	49.42 ± 0.04	23.99 ± 0.25	+51.5
S3 Bandwidth drop	Delivery ratio	0.867 ± 0.001	0.996 ± 0.001	+14.9
	Time degraded (s)	24.90 ± 0.00	0.51 ± 0.54	+98.0
	Mean latency (ms)	41.60 ± 0.16	21.80 ± 0.11	+47.6
S4 Packet loss increase	Delivery ratio	0.973 ± 0.001	0.997 ± 0.000	+2.5
	Time degraded (s)	23.58 ± 0.48	0.10 ± 0.23	+99.6
	Mean latency (ms)	20.13 ± 0.01	23.58 ± 0.20	-17.1
S5 Link failure	Delivery ratio	0.988 ± 0.001	0.996 ± 0.001	+0.8
	Recovery time (s)	1.15 ± 0.15	0.13 ± 0.14	+88.7
	Time degraded (s)	2.00 ± 0.12	0.74 ± 0.46	+63.0
	Mean latency (ms)	22.80 ± 0.01	25.02 ± 0.03	-9.8
S6 Congestion build-up	Delivery ratio	0.956 ± 0.000	0.997 ± 0.000	+4.4
	Time degraded (s)	19.46 ± 0.59	0.00 ± 0.00	+100.0
	Mean latency (ms)	57.21 ± 0.13	23.89 ± 0.27	+58.2
S7 Flapping link	Delivery ratio	0.938 ± 0.007	0.996 ± 0.001	+6.2
	Time degraded (s)	12.04 ± 0.86	0.74 ± 0.46	+93.9
	Route changes	12.0 ± 0.0	2.0 ± 0.0	+83.3
	Mean latency (ms)	20.59 ± 0.05	21.63 ± 0.06	-5.0
S8 Random faults	Delivery ratio	0.966 ± 0.039	0.996 ± 0.002	+3.1
	Mean latency (ms)	30.88 ± 10.43	23.37 ± 1.83	+24.3
S9 Lossy shortcut	Delivery ratio	0.924 ± 0.001	0.994 ± 0.000	+7.6
	Mean latency (ms)	21.50 ± 0.02	29.50 ± 0.11	-37.2

Every delivery-ratio difference in the fault scenarios S3 to S9 is statistically significant (Welch's t-test, $p < 0.05$; $p < 10^{-9}$ in all scenarios except random faults, where $p = 0.04$). In S1 and S2 the two policies delivered identical traffic ($p = 1$).

Table 5.4: Delivery ratio of all four policies, mean ± SD over 10 seeds

Scenario	IRP (proposed)	OSPF-like	Latency-only	Hop count
S1 Steady state	0.997 ± 0.000	0.997 ± 0.000	0.997 ± 0.000	0.997 ± 0.000
S2 Latency spike	0.997 ± 0.000	0.997 ± 0.000	0.997 ± 0.000	0.997 ± 0.000
S3 Bandwidth drop	0.996 ± 0.001	0.867 ± 0.001	0.996 ± 0.001	0.867 ± 0.001
S4 Packet loss increase	0.997 ± 0.000	0.973 ± 0.001	0.973 ± 0.001	0.948 ± 0.001
S5 Link failure	0.996 ± 0.001	0.988 ± 0.001	0.988 ± 0.001	0.978 ± 0.002
S6 Congestion build-up	0.997 ± 0.000	0.956 ± 0.000	0.997 ± 0.000	0.852 ± 0.001
S7 Flapping link	0.996 ± 0.001	0.938 ± 0.007	0.939 ± 0.007	0.988 ± 0.001
S8 Random faults	0.996 ± 0.002	0.966 ± 0.039	0.986 ± 0.013	0.959 ± 0.056
S9 Lossy shortcut	0.994 ± 0.000	0.924 ± 0.001	0.924 ± 0.001	0.924 ± 0.001

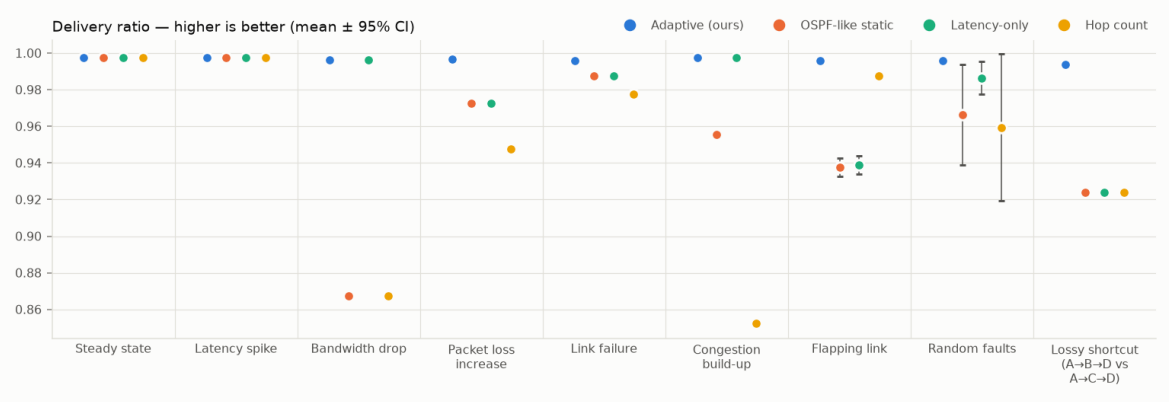


Fig. 5.1: Delivery ratio per scenario and policy

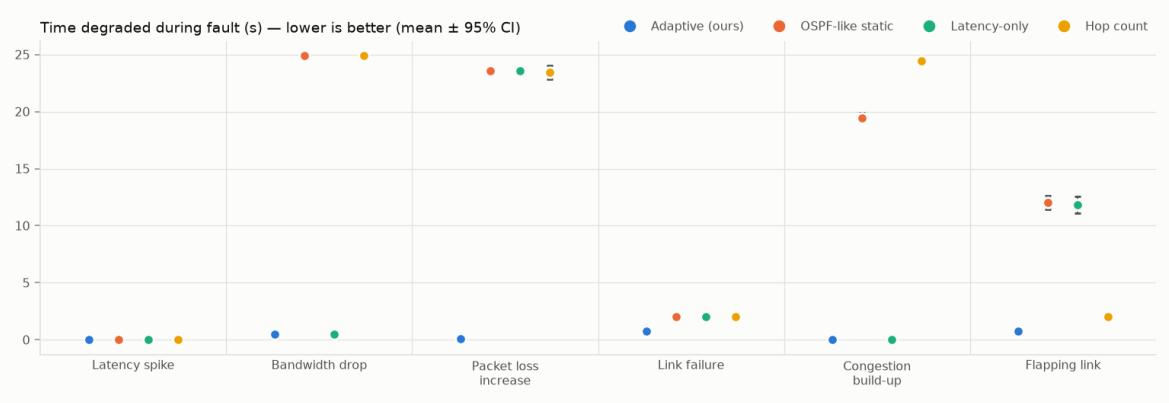


Fig. 5.2: Time with degraded service during the fault

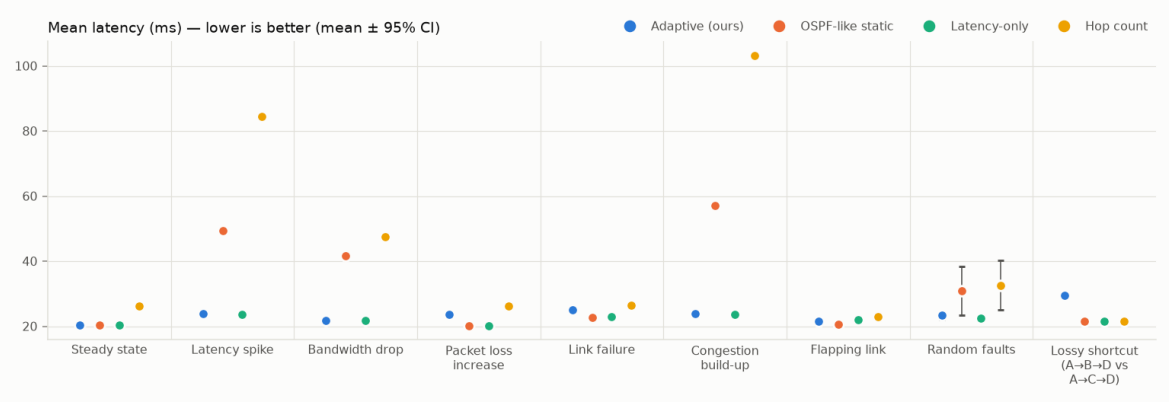


Fig. 5.3: Mean latency per scenario and policy

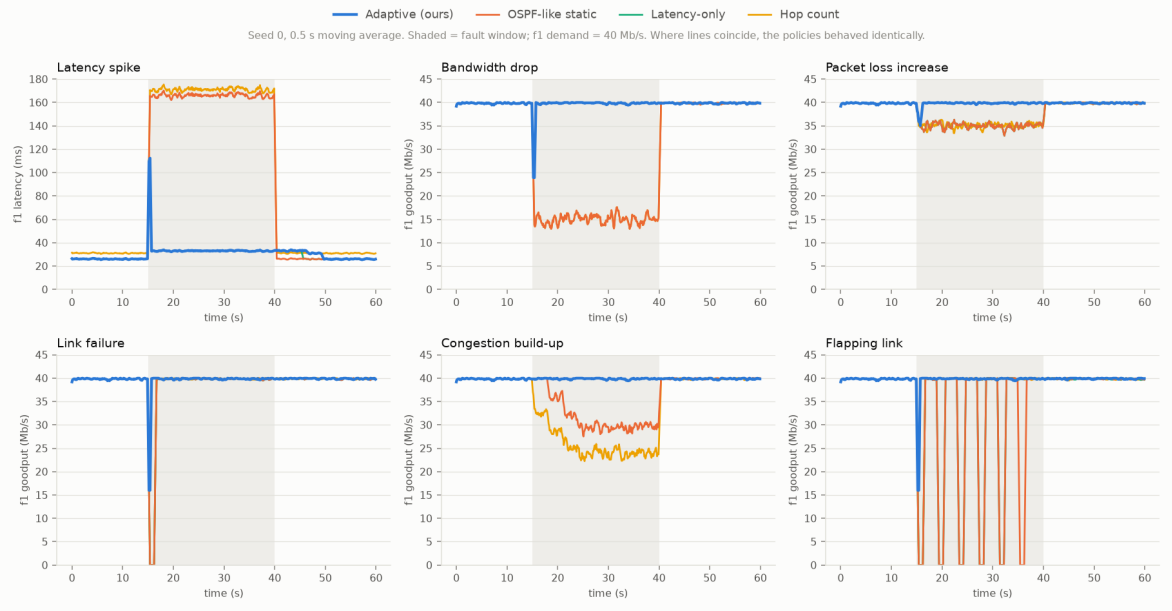


Fig. 5.4: Throughput of flow f1 over time (blue: IRP, orange: OSPF-like; grey band: fault window)

5.4 Analysis and Discussion

Reliability. IRP kept the delivery ratio at 0.994 or higher in every scenario, while the OSPF-like baseline fell to 0.867 under a bandwidth drop, 0.938 on a flapping link and 0.924 on the lossy shortcut. The reason is that OSPF keeps a link as long as it is up: under a bandwidth drop E-H is still up but can carry only 15 of the 40 Mb/s that f1 needs, so the excess is dropped for the whole 25 s fault (24.9 s degraded). IRP's predicted utilisation for E-H exceeds 1, its overflow-loss term rises, and it moves f1 to A→C→F→H or A→D→G→H within one or two probe rounds (0.51 s degraded on average).

Recovery speed. On a link failure IRP recovered in 0.13 s against 1.15 s for the baseline, about nine times faster, even though both use the same three-probe dead timer. The first missed probe already pushes the loss estimate to about 30 %, which makes the current path's cost exceed the 15 % hysteresis margin, so IRP moves before the dead timer fires. The route-change log records this as "loss on B-E degraded: loss ≈30.2%" at 15.3 s, while OSPF switches only at 16.3 s.

Stability. On the flapping link the baseline changed route 12 times, following every down and up transition, and was degraded for 12.0 s. IRP changed route only twice: the flap penalty keeps E-H expensive for tens of seconds after each failure, so f1 stays on the stable alternative. The latency-only policy was even worse than OSPF here (19.4 changes), showing that adaptivity without damping is harmful.

Why multiple metrics matter. The latency-only baseline matched IRP on latency spikes, bandwidth drops and congestion, because those faults all appear as delay or queueing. It failed

exactly where IRP's extra terms act: on packet loss (0.973), flapping (0.939) and the lossy shortcut (0.924), which do not raise delay.

Trade-off. In four scenarios IRP had a higher mean latency than OSPF: loss increase (23.6 vs 20.1 ms), link failure (25.0 vs 22.8 ms), flapping (21.6 vs 20.6 ms) and the lossy shortcut (29.5 vs 21.5 ms). This is intended: IRP pays a few milliseconds to use a clean path, and it returns to a recovered link only after the hold-down and flap penalty allow it. The weights control this balance. Raising w_lat favours delay and raising w_loss favours reliability.

5.5 Critical Evaluation

Strengths. IRP is significantly more reliable than the static baseline in all seven loss, capacity, failure, congestion and flapping scenarios, halves the latency under a latency spike, recovers about nine times faster from failures, and resists flapping. It is identical to the baseline on a healthy network, so adaptation costs nothing when there are no faults. It reuses Dijkstra unchanged, explains every decision, and is backed by 29 tests and 360 seeded runs.

Weaknesses. It accepts higher latency in loss-driven scenarios, and it makes more route changes than OSPF in the latency-spike and congestion scenarios (2.6 and 3.6 on average against 0). The weights are hand-tuned and may need retuning for other topologies or traffic. Probing every link every 0.5 s adds measurement overhead that a static protocol does not have.

Conditions where the proposed scheme fails or gives no gain. (1) When every path is degraded in the same way there is nothing better to switch to, so IRP behaves like the baseline. (2) Very short faults (shorter than about one probe interval) are over before the monitor can react. (3) Faults that change faster than the 3 s hold-down cannot be followed for cost-driven changes. (4) With many flows choosing paths at the same time from the same view, flows could move together onto the same alternative; own-load subtraction handles a flow's own traffic but not the simultaneous decisions of other flows. (5) In the latency-spike scenario IRP gives no reliability gain over the latency-only baseline, because the fault affects delay alone.

6. CONCLUSION AND FUTURE WORK

6.1 Summary of Work and Key Findings

This project designed and implemented an Intelligent Routing Protocol that chooses paths from live measurements of delay, loss, free bandwidth and stability, combined into one additive cost and paired with hysteresis, hold-down, flap damping, own-load subtraction and early failure detection. It was evaluated against three baselines over nine scenarios with ten seeds each. Compared with OSPF-like routing:

- Delivery ratio was significantly higher in all seven fault scenarios, for example 0.996 vs 0.867 (+14.9 %) under a bandwidth drop, 0.997 vs 0.956 under congestion and 0.994 vs 0.924 on the lossy shortcut.
- Recovery from a link failure took 0.13 s instead of 1.15 s (about 9× faster).
- Time with degraded service fell from 24.9 s to 0.5 s (bandwidth drop), from 23.6 s to 0.1 s (loss increase) and from 19.5 s to 0 s (congestion).
- On a flapping link IRP made 2 route changes instead of 12 and delivered 99.6 % of packets instead of 93.8 %.
- On a healthy network IRP behaved identically to the baseline (0.997 delivery, 20.3 ms latency).
- The cost was 1 to 8 ms of extra mean latency in four scenarios where a longer, loss-free path was chosen.

6.2 Limitations

- Flow-level, discrete-time simulation with an M/M/1 queueing approximation; no packet-level or TCP behaviour.
- Routes are computed per flow from a shared link-state view; LSA flooding and distributed convergence are not modelled.
- Cost weights and stability parameters were tuned by hand on the scenario suite.
- Probing overhead and its effect on link capacity are not accounted for.

6.3 Future Enhancements

- Learn the cost weights automatically, for example with Bayesian optimisation over the scenario suite.

- Validate the protocol with real packets in a packet-level emulator such as Mininet or ns-3.
- Turn it into a fully distributed protocol in which routers flood their measurements as link-state advertisements.
- Add multipath load splitting across near-equal-cost paths and coordination between flows that react to the same event.

7. REFERENCES

[1] J. Moy, "OSPF Version 2," RFC 2328, IETF, Apr. 1998.

[2] C. Hedrick, "Routing Information Protocol," RFC 1058, IETF, Jun. 1988.

[3] E. W. Dijkstra, "A note on two problems in connexion with graphs," Numerische Mathematik, vol. 1, no. 1, pp. 269–271, 1959.

[4] A. Khanna and J. Zinky, "The revised ARPANET routing metric," in Proc. ACM SIGCOMM, Austin, TX, USA, 1989, pp. 45–56.

[5] Z. Wang and J. Crowcroft, "Analysis of shortest-path routing algorithms in a dynamic network environment," ACM SIGCOMM Comput. Commun. Rev., vol. 22, no. 2, pp. 63–71, 1992.

[6] B. Fortz and M. Thorup, "Internet traffic engineering by optimizing OSPF weights," in Proc. IEEE INFOCOM, Tel Aviv, Israel, 2000, pp. 519–528.

[7] D. S. J. De Couto, D. Aguayo, J. Bicket, and R. Morris, "A high-throughput path metric for multi-hop wireless routing," in Proc. ACM MobiCom, San Diego, CA, USA, 2003, pp. 134–146.

[8] S. Kandula, D. Katabi, B. Davie, and A. Charny, "Walking the tightrope: Responsive yet stable traffic engineering," in Proc. ACM SIGCOMM, Philadelphia, PA, USA, 2005, pp. 253–264.

[9] D. Andersen, H. Balakrishnan, F. Kaashoek, and R. Morris, "Resilient overlay networks," in Proc. 18th ACM Symp. Operating Systems Principles (SOSP), Banff, Canada, 2001, pp. 131–145.

[10] J. A. Boyan and M. L. Littman, "Packet routing in dynamically changing networks: A reinforcement learning approach," in Advances in Neural Information Processing Systems 6, 1994, pp. 671–678.

[11] C. Villamizar, R. Chandra, and R. Govindan, "BGP route flap damping," RFC 2439, IETF, Nov. 1998.

[12] D. Katz and D. Ward, "Bidirectional Forwarding Detection (BFD)," RFC 5880, IETF, Jun. 2010.

[13] D. Savage, J. Ng, S. Moore, D. Slice, P. Paluch, and R. White, "Cisco's Enhanced Interior Gateway Routing Protocol (EIGRP)," RFC 7868, IETF, May 2016.

[14] L. Kleinrock, Queueing Systems, Volume 1: Theory. New York, NY, USA: Wiley, 1975.

[15] B. L. Welch, "The generalization of 'Student's' problem when several different population variances are involved," Biometrika, vol. 34, no. 1–2, pp. 28–35, 1947.

APPENDIX A: IMPORTANT SOURCE CODE AND CONFIGURATION LISTINGS

The complete source code is in the submitted archive F2_Team18_SourceCode.zip and at https://github.com/nehadamani0909/F2_Team18.

A.1 Adaptive link cost (irp/policies.py)

```
def link_cost(self, link, est, ctx):
    w = self.weights
    rho = _predicted_utilisation(est, ctx)
    delay = est.latency_ms + queue_delay_ms(rho)
    p = 1.0 - (1.0 - min(est.loss, 0.999)) * (1.0 - overflow_loss(rho))
    p = min(p, 0.999)
    available = max(est.capacity_mbps - max(0.0, est.load_mbps - ctx.own_load_mbps), 1e-3)
    parts = {
        "delay": w.latency * delay,
        "loss": w.loss * -math.log(1.0 - p),
        "bandwidth": w.bandwidth * w.ref_bw_mbps / available,
        "stability": w.stability * est.flap_score,
    }
    return sum(parts.values()), parts
```

A.2 Route-update decision (irp/engine.py)

```
def _update_flow(self, t: float, flow: Flow) -> RouteChange | None:
    cur = self.routes.get(flow.id)
    best, best_cost = self.shortest_path(flow.src, flow.dst, flow)
    if flow.id not in self.routes:
        return self._install(t, flow, None, best, INF, best_cost, "initial route")

    cur_cost = self.path_cost(cur, flow)
    self.costs[flow.id] = cur_cost
    if best == cur:
        self._refresh_baseline(flow, cur)
        return None

    if math.isinf(cur_cost):
        reason = self._failure_reason(cur) if cur else "destination reachable again"
        return self._install(t, flow, cur, best, cur_cost, best_cost, reason)

    if best is None or best_cost >= cur_cost - 1e-9:
        return None
    improvement = (cur_cost - best_cost) / cur_cost
    if improvement <= self.policy.hysteresis:
        return None
    if t - self.last_change.get(flow.id, -INF) < self.policy.hold_down_s:
        return None
    reason = self._degradation_reason(flow, cur, improvement, cur_cost - best_cost)
    return self._install(t, flow, cur, best, cur_cost, best_cost, reason)
```

A.3 Early failure detection in the link monitor (irp/monitor.py)

```
if not s.up:
    e.missed += 1
    # Every probe was lost: that is evidence of loss too.
    e.loss = self._ewma(e.loss, 1.0)
    if e.up and e.missed >= self.dead_after:
        e.up = False
        e.flaps += 1
        e.flap_score += 1.0
        changed.append(lid)
    continue
```

A.4 Commands to reproduce the results

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python -m irp demo                            # A-B-D example and a link failure,
OSPF vs IRP
python -m irp scenarios                       # list the nine experiments
python -m irp run loss_increase -p adaptive  # one experiment with its route-
change log
python -m irp evaluate --seeds 10             # full evaluation -> results/
python -m irp dashboard                       # dashboard at http://localhost:8501
pytest -q                                     # 29 tests
```

APPENDIX B: ADDITIONAL SCREENSHOTS AND RAW RESULT TABLES

B.1 Dashboard screenshots



Fig. B.1: Dashboard: steady-state experiment

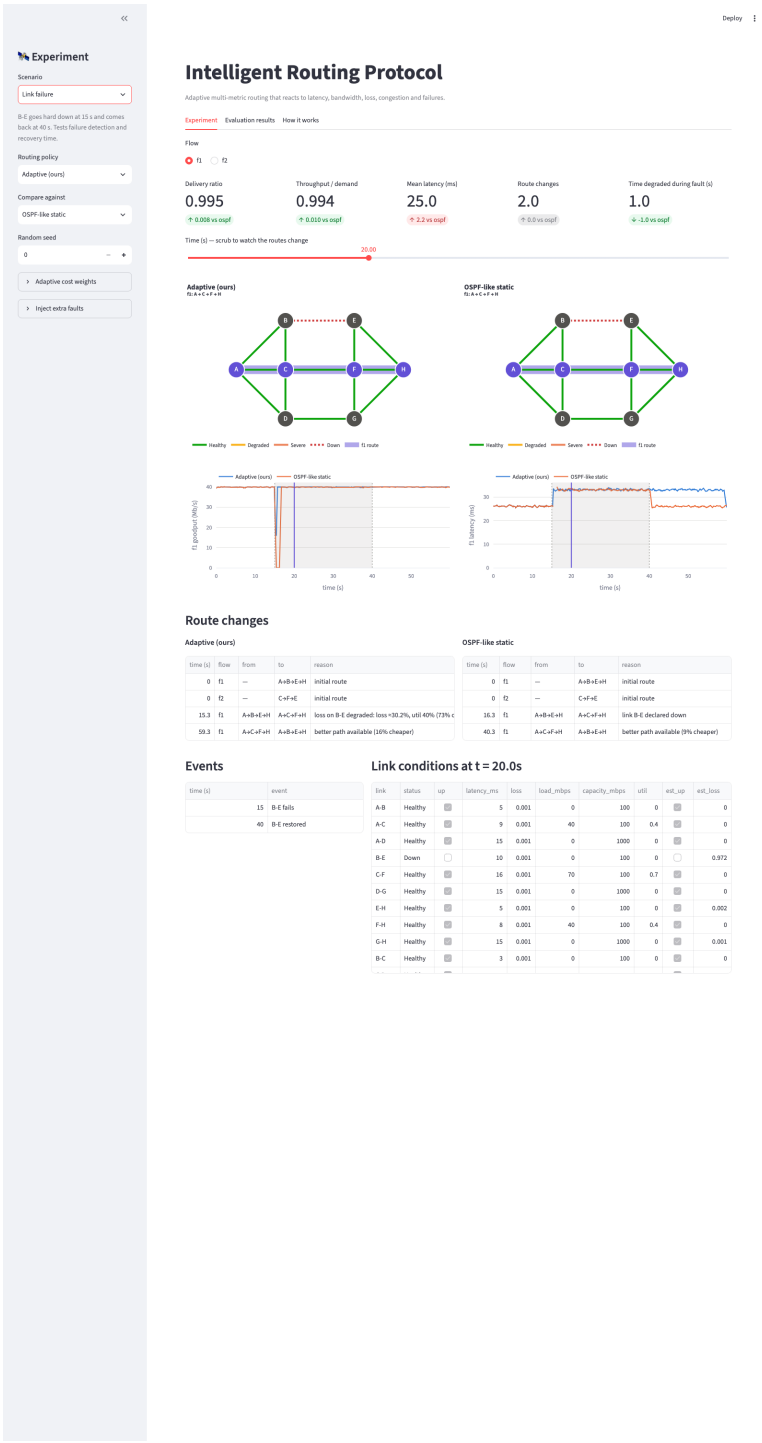


Fig. B.2: Dashboard: link-failure experiment

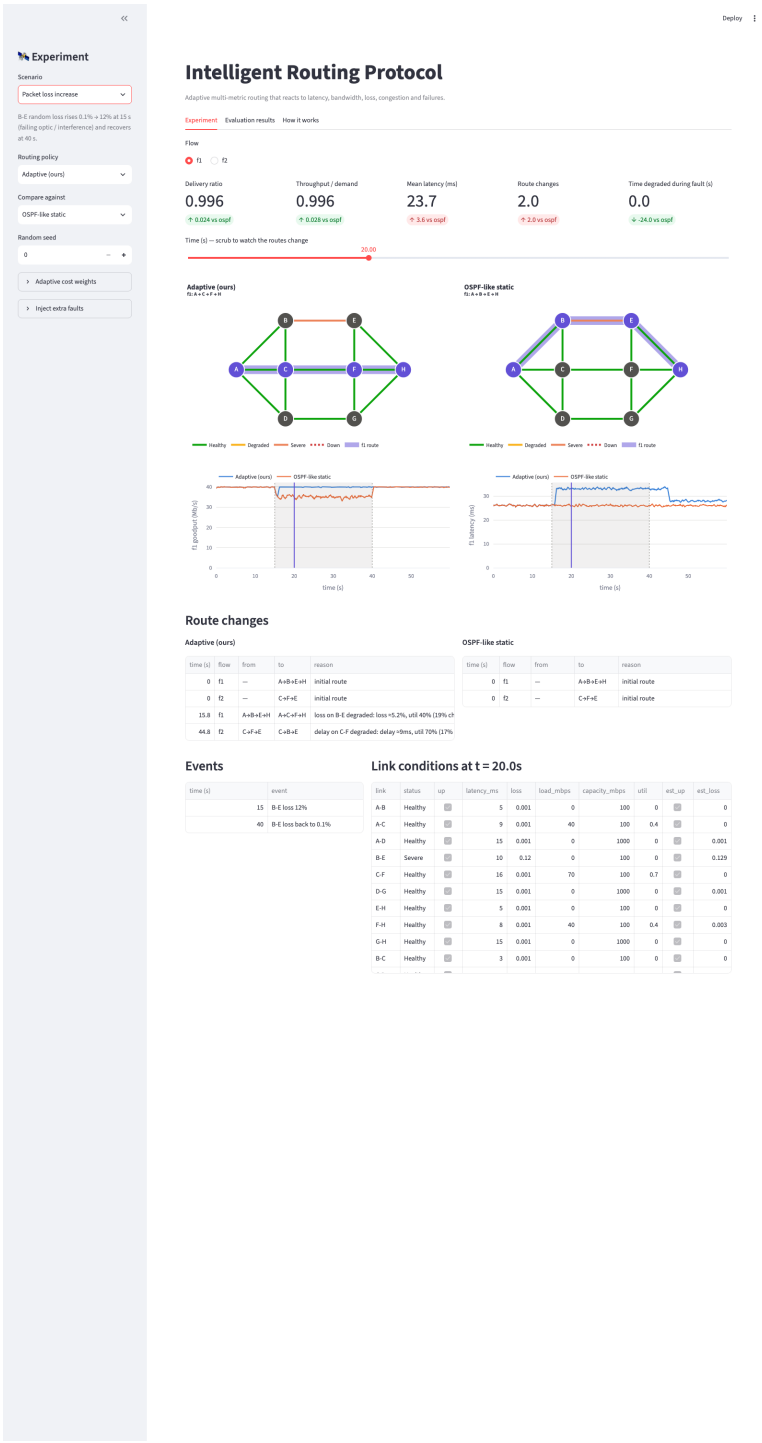


Fig. B.3: Dashboard: packet-loss experiment

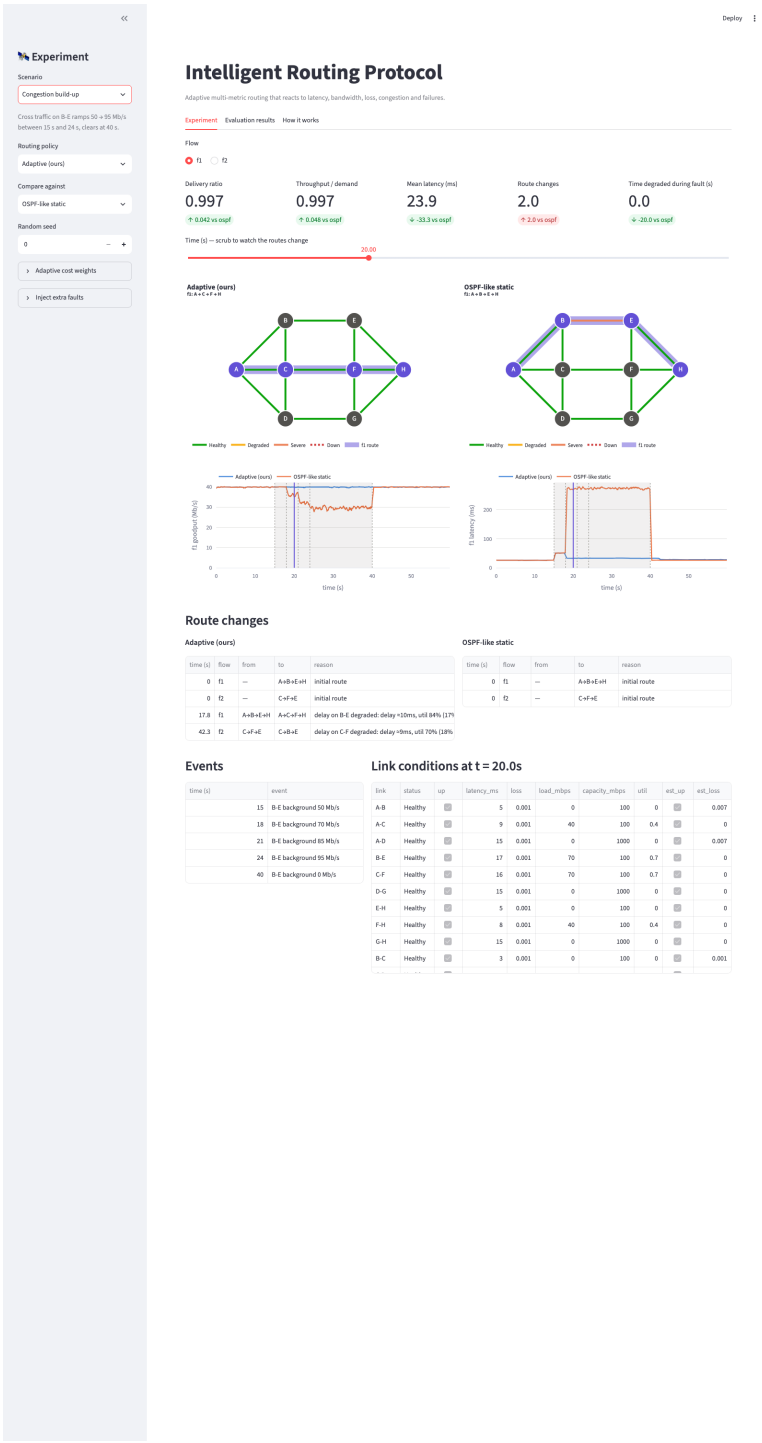


Fig. B.4: Dashboard: congestion experiment

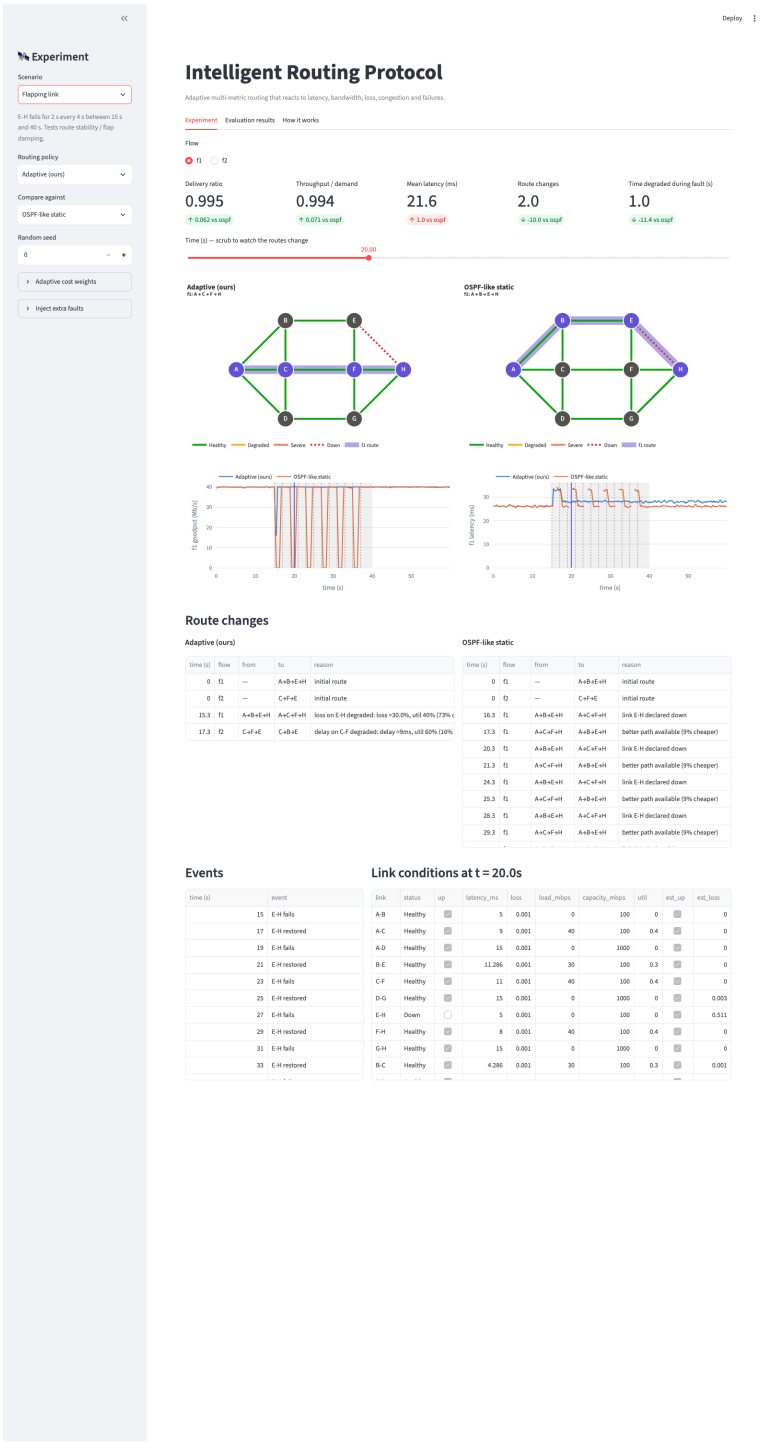


Fig. B.5: Dashboard: flapping-link experiment

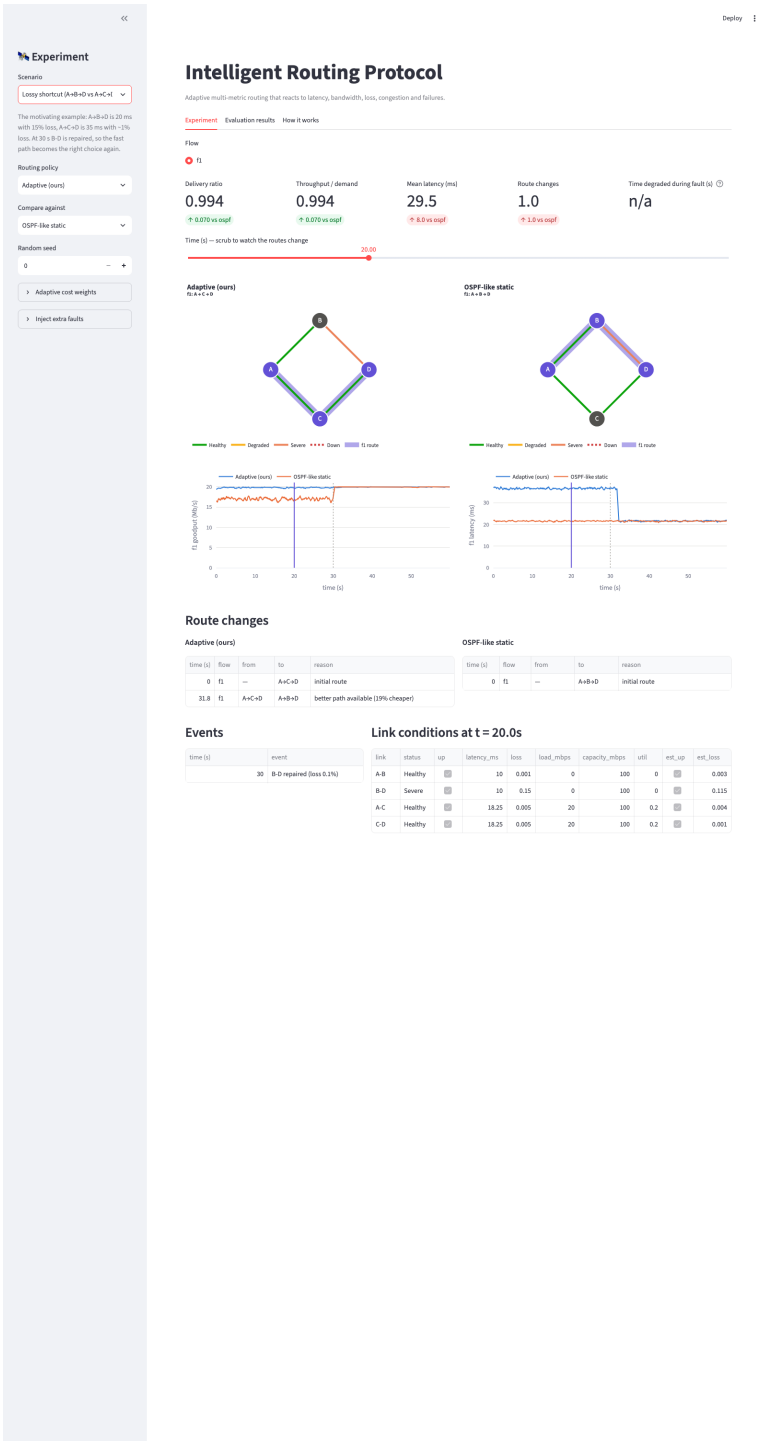
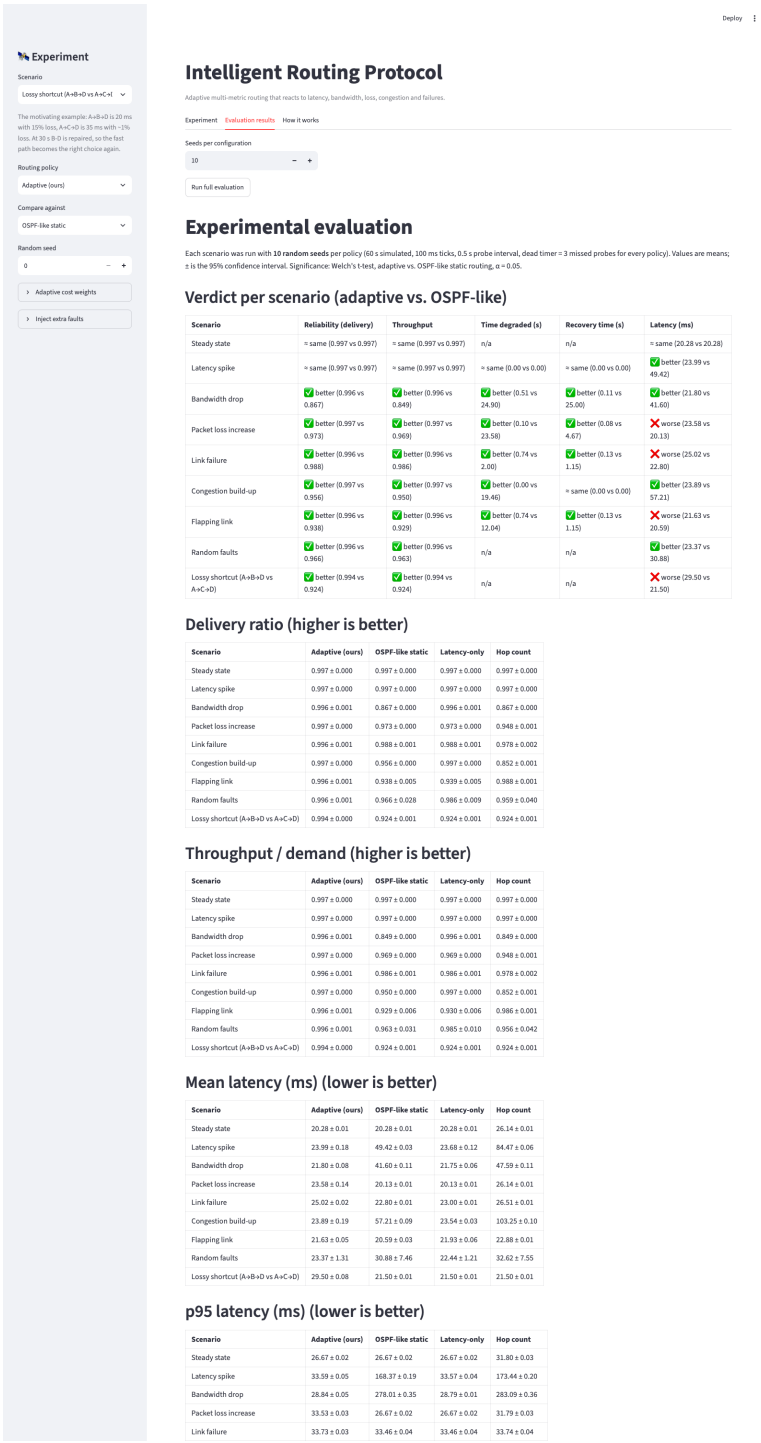


Fig. B.6: Dashboard: lossy-shortcut experiment



Experimental evaluation

Each scenario was run with **10 random seeds** per policy (60 s simulated, 100 ms ticks, 0.5 s probe interval, dead timer = 3 missed probes for every policy). Values are means; ± is the 95% confidence interval. Significance: Welch's t-test, adaptive vs. OSPF-like static routing, α = 0.05.

Verdict per scenario (adaptive vs. OSPF-like)

Scenario	Reliability (delivery)	Throughput	Time degraded (s)	Recovery time (s)	Latency (ms)
Steady state	≈ same (0.997 vs 0.997)	≈ same (0.997 vs 0.997)	n/a	n/a	≈ same (20.28 vs 20.28)
Latency spike	≈ same (0.997 vs 0.997)	≈ same (0.997 vs 0.997)	≈ same (0.00 vs 0.00)	≈ same (0.00 vs 0.00)	✅ better (23.99 vs 49.42)
Bandwidth drop	✅ better (0.996 vs 0.867)	✅ better (0.996 vs 0.849)	✅ better (0.51 vs 24.90)	✅ better (0.11 vs 25.00)	✅ better (21.80 vs 41.60)
Packet loss increase	✅ better (0.997 vs 0.973)	✅ better (0.997 vs 0.969)	✅ better (0.10 vs 23.58)	✅ better (0.08 vs 4.67)	❌ worse (23.58 vs 20.13)
Link failure	✅ better (0.996 vs 0.988)	✅ better (0.996 vs 0.986)	✅ better (0.74 vs 2.00)	✅ better (0.13 vs 1.15)	❌ worse (25.02 vs 22.80)
Congestion build-up	✅ better (0.997 vs 0.956)	✅ better (0.997 vs 0.950)	✅ better (0.00 vs 19.46)	≈ same (0.00 vs 0.00)	✅ better (23.89 vs 57.21)
Flapping link	✅ better (0.996 vs 0.938)	✅ better (0.996 vs 0.929)	✅ better (0.74 vs 12.04)	✅ better (0.13 vs 1.15)	❌ worse (21.63 vs 20.59)
Random faults	✅ better (0.996 vs 0.966)	✅ better (0.996 vs 0.963)	n/a	n/a	✅ better (23.37 vs 30.88)
Lossy shortcut (A→B→D vs A→C→D)	✅ better (0.994 vs 0.924)	✅ better (0.994 vs 0.924)	n/a	n/a	❌ worse (29.50 vs 21.50)

Delivery ratio (higher is better)

Scenario	Adaptive (ours)	OSPF-like static	Latency-only	Hop count
Steady state	0.997 ± 0.000	0.997 ± 0.000	0.997 ± 0.000	0.997 ± 0.000
Latency spike	0.997 ± 0.000	0.997 ± 0.000	0.997 ± 0.000	0.997 ± 0.000
Bandwidth drop	0.996 ± 0.001	0.867 ± 0.000	0.996 ± 0.001	0.867 ± 0.000
Packet loss increase	0.997 ± 0.000	0.973 ± 0.000	0.973 ± 0.000	0.948 ± 0.001
Link failure	0.996 ± 0.001	0.988 ± 0.001	0.988 ± 0.001	0.978 ± 0.002
Congestion build-up	0.997 ± 0.000	0.956 ± 0.000	0.997 ± 0.000	0.852 ± 0.001
Flapping link	0.996 ± 0.001	0.938 ± 0.005	0.939 ± 0.005	0.988 ± 0.001
Random faults	0.996 ± 0.001	0.966 ± 0.028	0.986 ± 0.009	0.959 ± 0.040
Lossy shortcut (A→B→D vs A→C→D)	0.994 ± 0.000	0.924 ± 0.001	0.924 ± 0.001	0.924 ± 0.001

Throughput / demand (higher is better)

Scenario	Adaptive (ours)	OSPF-like static	Latency-only	Hop count
Steady state	0.997 ± 0.000	0.997 ± 0.000	0.997 ± 0.000	0.997 ± 0.000
Latency spike	0.997 ± 0.000	0.997 ± 0.000	0.997 ± 0.000	0.997 ± 0.000
Bandwidth drop	0.996 ± 0.001	0.849 ± 0.000	0.996 ± 0.001	0.849 ± 0.000
Packet loss increase	0.997 ± 0.000	0.969 ± 0.000	0.969 ± 0.000	0.948 ± 0.001
Link failure	0.996 ± 0.001	0.986 ± 0.001	0.986 ± 0.001	0.978 ± 0.002
Congestion build-up	0.997 ± 0.000	0.950 ± 0.000	0.997 ± 0.000	0.852 ± 0.001
Flapping link	0.996 ± 0.001	0.929 ± 0.006	0.930 ± 0.006	0.986 ± 0.001
Random faults	0.996 ± 0.001	0.963 ± 0.031	0.985 ± 0.010	0.956 ± 0.042
Lossy shortcut (A→B→D vs A→C→D)	0.994 ± 0.000	0.924 ± 0.001	0.924 ± 0.001	0.924 ± 0.001

Mean latency (ms) (lower is better)

Scenario	Adaptive (ours)	OSPF-like static	Latency-only	Hop count
Steady state	20.28 ± 0.01	20.28 ± 0.01	20.28 ± 0.01	26.14 ± 0.01
Latency spike	23.99 ± 0.18	49.42 ± 0.03	23.68 ± 0.12	84.47 ± 0.06
Bandwidth drop	21.80 ± 0.08	41.60 ± 0.11	21.75 ± 0.06	47.59 ± 0.11
Packet loss increase	23.58 ± 0.14	20.13 ± 0.01	20.13 ± 0.01	26.14 ± 0.01
Link failure	25.02 ± 0.02	22.80 ± 0.01	23.00 ± 0.01	26.51 ± 0.01
Congestion build-up	23.89 ± 0.19	57.21 ± 0.09	23.54 ± 0.03	103.25 ± 0.10
Flapping link	21.63 ± 0.05	20.59 ± 0.03	21.93 ± 0.06	22.88 ± 0.01
Random faults	23.37 ± 1.31	30.88 ± 7.46	22.44 ± 1.21	32.62 ± 7.55
Lossy shortcut (A→B→D vs A→C→D)	29.50 ± 0.08	21.50 ± 0.01	21.50 ± 0.01	21.50 ± 0.01

p95 latency (ms) (lower is better)

Scenario	Adaptive (ours)	OSPF-like static	Latency-only	Hop count
Steady state	26.67 ± 0.02	26.67 ± 0.02	26.67 ± 0.02	31.80 ± 0.03
Latency spike	33.59 ± 0.05	168.37 ± 0.19	33.57 ± 0.04	173.44 ± 0.20
Bandwidth drop	28.84 ± 0.05	278.01 ± 0.35	28.79 ± 0.01	283.09 ± 0.36
Packet loss increase	33.53 ± 0.03	26.67 ± 0.02	26.67 ± 0.02	31.79 ± 0.03
Link failure	33.73 ± 0.03	33.46 ± 0.04	33.46 ± 0.04	33.74 ± 0.04

Fig. B.7: Dashboard: evaluation results page

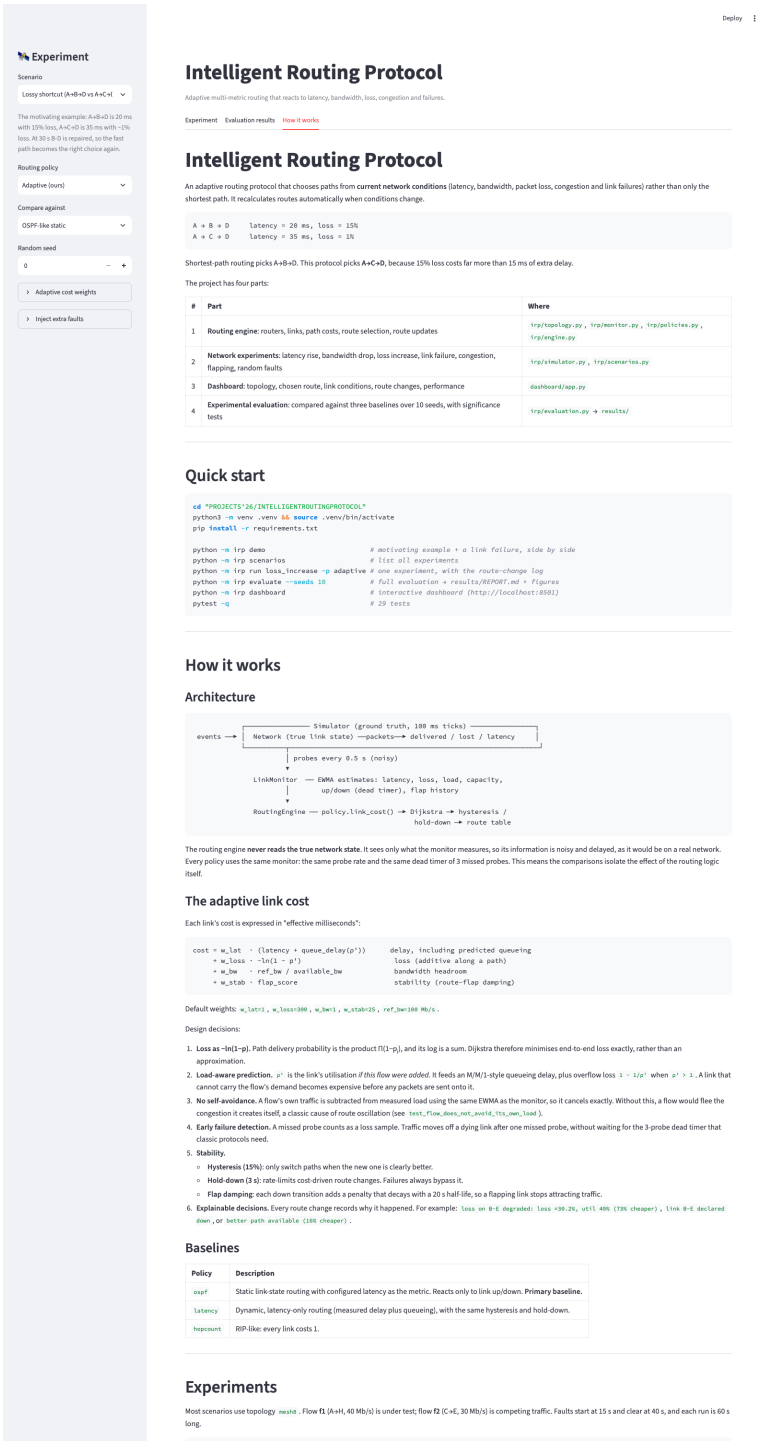


Fig. B.8: Dashboard: how-it-works page

B.2 Route-change log and metrics (python -m irp demo)

```
== Path choice on the motivating example ==
   A->B->D : 20 ms, 15% loss       A->C->D : 35 ms, ~1% loss

  ospf      chooses A->B->D  (path cost 20.0)
  adaptive  chooses A->C->D  (path cost 41.5)
      A-B:   12.1  (delay=10.8, loss=0.3, bandwidth=1.0, stability=0.0)
      B-D:   60.5  (delay=10.8, loss=48.8, bandwidth=1.0, stability=0.0)
      A-C:   20.8  (delay=18.2, loss=1.5, bandwidth=1.0, stability=0.0)
      C-D:   20.8  (delay=18.2, loss=1.5, bandwidth=1.0, stability=0.0)
```

```
== Link failure: B-E fails at 15 s, restored at 40 s ==

  [ospf]
    t=   0.0s  f1: (none)  =>  A->B->E->H   [initial route]
    t=   0.0s  f2: (none)  =>  C->F->E   [initial route]
    t=  16.3s  f1: A->B->E->H  =>  A->C->F->H   [link B-E declared down]
    t=  40.3s  f1: A->C->F->H  =>  A->B->E->H   [better path available (9% cheaper)]
    delivery_ratio           0.986
    mean_latency_ms          22.790
    p95_latency_ms           33.557
    avg_throughput_mbps      68.929
    throughput_ratio         0.985
    route_changes            2
    recovery_time_s          1.200
    recovered                True
    degraded_s               2.000
    fault_throughput_ratio   0.945
    fault_delivery_ratio     0.945
    fault_mean_latency_ms    32.993

  [adaptive]
    t=   0.0s  f1: (none)  =>  A->B->E->H   [initial route]
    t=   0.0s  f2: (none)  =>  C->F->E   [initial route]
    t=  15.3s  f1: A->B->E->H  =>  A->C->F->H   [loss on B-E degraded: loss ~30.2%, util 40%
(73% cheaper)]
    t=  59.3s  f1: A->C->F->H  =>  A->B->E->H   [better path available (16% cheaper)]
    delivery_ratio           0.995
    mean_latency_ms          24.959
    p95_latency_ms           33.721
    avg_throughput_mbps      69.606
    throughput_ratio         0.994
    route_changes            2
    recovery_time_s          0.200
    recovered                True
    degraded_s               1.000
    fault_throughput_ratio   0.985
    fault_delivery_ratio     0.985
    fault_mean_latency_ms    33.075
```

B.3 Additional raw results

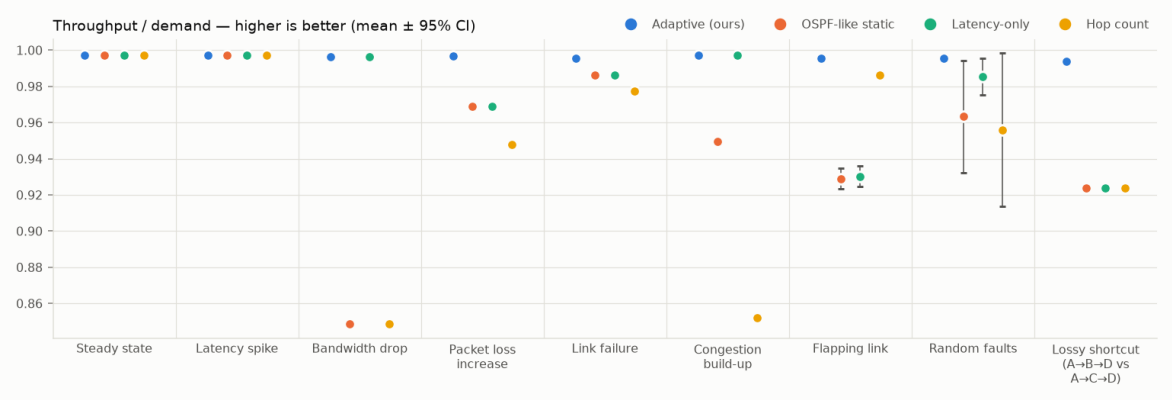


Fig. B.9: Throughput ratio per scenario and policy

Table B.1: Mean latency (ms) of all four policies, mean ± SD over 10 seeds

Scenario	IRP (proposed)	OSPF-like	Latency-only	Hop count
S1 Steady state	20.28 ± 0.02	20.28 ± 0.02	20.28 ± 0.02	26.14 ± 0.02
S2 Latency spike	23.99 ± 0.25	49.42 ± 0.04	23.68 ± 0.17	84.47 ± 0.08
S3 Bandwidth drop	21.80 ± 0.11	41.60 ± 0.16	21.75 ± 0.09	47.59 ± 0.16
S4 Packet loss increase	23.58 ± 0.20	20.13 ± 0.01	20.13 ± 0.01	26.14 ± 0.02
S5 Link failure	25.02 ± 0.03	22.80 ± 0.01	23.00 ± 0.02	26.51 ± 0.01

Scenario	IRP (proposed)	OSPF-like	Latency-only	Hop count
S6 Congestion build-up	23.89 ± 0.27	57.21 ± 0.13	23.54 ± 0.04	103.25 ± 0.14
S7 Flapping link	21.63 ± 0.06	20.59 ± 0.05	21.93 ± 0.09	22.88 ± 0.02
S8 Random faults	23.37 ± 1.83	30.88 ± 10.43	22.44 ± 1.69	32.62 ± 10.56
S9 Lossy shortcut	29.50 ± 0.11	21.50 ± 0.02	21.50 ± 0.02	21.50 ± 0.02

Table B.2: Route changes of all four policies, mean ± SD over 10 seeds

Scenario	IRP (proposed)	OSPF-like	Latency-only	Hop count
S1 Steady state	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0
S2 Latency spike	2.6 ± 1.0	0.0 ± 0.0	2.0 ± 0.0	0.0 ± 0.0
S3 Bandwidth drop	2.0 ± 0.0	0.0 ± 0.0	2.0 ± 0.0	0.0 ± 0.0
S4 Packet loss increase	2.2 ± 0.6	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0
S5 Link failure	1.1 ± 0.3	2.0 ± 0.0	4.0 ± 0.0	2.0 ± 0.0
S6 Congestion build-up	3.6 ± 2.1	0.0 ± 0.0	4.0 ± 0.0	0.0 ± 0.0
S7 Flapping link	2.0 ± 0.0	12.0 ± 0.0	19.4 ± 2.8	1.0 ± 0.0
S8 Random faults	3.5 ± 1.8	0.6 ± 1.0	4.5 ± 3.6	0.2 ± 0.4
S9 Lossy shortcut	1.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0

Raw per-seed data for all 360 runs is in results/raw_runs.csv, and the full statistical report with 95 % confidence intervals and p-values is in results/REPORT.md.